Experiment no:10

Aim:

To deploy a MERN (MongoDB, Express.js, React.js, Node.js) Stack Web Application on a cloud platform and make it accessible over the internet.

Theory:

MERN Stack

MERN is a full-stack JavaScript framework consisting of:

Technology	Purpose
MongoDB	Database
Express.js	Backend Framework
React.js	Frontend Library
Node.js	Runtime Environment

Advantages of MERN Stack

- Uses JavaScript throughout the application
- Scalable and flexible
- Fast development process
- Efficient client-server communication

Cloud Deployment

Cloud deployment refers to hosting an application on remote servers so that it can be accessed from anywhere through the internet.

Benefits

- High availability
- Scalability
- Reliability
- Easy maintenance
- Global accessibility

Popular Cloud Platforms

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud
- Render
- Vercel

Software Requirements

- Node.js
- MongoDB Atlas Account
- React Application
- Express.js Backend
- Git
- Cloud Platform Account (Render/Vercel/AWS)

Architecture

React Frontend

Cloud Frontend Hosting

Express + Node.js Backend

MongoDB Atlas Database

Procedure:

Step 1: Create MongoDB Atlas Database

1. Sign up for MongoDB Atlas.
2. Create a cluster.
3. Create a database user.
4. Obtain the connection string.

Example:

mongodb+srv://username:password@cluster.mongodb.net/studentdb

Step 2: Prepare Backend Application

Install Dependencies

npm install express mongoose cors dotenv

Sample Server Code

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());
mongoose.connect(process.env.MONGO_URI);
app.get("/", (req, res) => {
  res.send("MERN Application Running");
});
app.listen(5000, () => {
  console.log("Server Started");
});
```

Step 3: Create Environment Variables

Create a .env file:

```
MONGO_URI=mongodb+srv://username:password@cluster.mongodb.net/studentdb
PORT=5000
```

Step 4: Build React Application

Navigate to the React project folder:

```
npm run build
```

Output:

```
build/
```

The build folder contains optimized production files.

Step 5: Push Project to GitHub

Initialize Git:

```
git init
git add .
git commit -m "Initial MERN Deployment"
```

Create a repository on:

GitHub Official Website

Push code:

git remote add origin <repository-url>

git push -u origin main

Step 6: Deploy Backend on Render

1. Create a Render account.
2. Connect GitHub repository.
3. Create a new Web Service.
4. Select the repository.
5. Configure:
    - Build Command:

npm install

- Start Command:

node server.js

6. Add environment variables.
7. Deploy the application.

Step 7: Deploy Frontend

Frontend can be deployed using:

Option 1: Vercel

1. Import React project from GitHub.
2. Configure build settings.
3. Deploy.

Option 2: Render Static Site

1. Create Static Site.
2. Connect repository.
3. Configure:

Build Command: npm run build

Publish Directory: build

4. Deploy.

Testing the Deployment

Backend URL

https://your-backend.onrender.com

Response

MERN Application Running

Frontend URL

https://your-app.vercel.app

Output:

Frontend Successfully Hosted

Backend Successfully Hosted

MongoDB Atlas Connected

Application Accessible Through Public URL

Conclusion:

The MERN Stack Web Application was successfully deployed on a cloud platform. The frontend, backend, and MongoDB database were integrated and made accessible through public URLs.

R1 (5 Marks)	R2 (5 Marks)	R3 (5 Marks)	Total (15 Marks)	Signature